PRECOG RECRUITMENT TASKS 2026

Tasks are centered around four central themes

- NLP
- Computer Vision
- Graphs
- Quant

Applicants must choose any one theme and complete the task listed under that theme.

You are encouraged to draw upon existing literature, blogs, and other credible resources to inform your design decisions. While doing so, you are expected to clearly justify the choices and assumptions you make.

You are not required to follow the architectures or methods suggested in the task description, if any. Feel free to design and experiment with your own approaches, modify existing ideas, combine techniques, or explore entirely new directions.

Importantly, all experiments are valuable, including those that do not yield positive results. Any approach you attempt, successful or not should be documented and discussed in your report or presentation.

We strongly recommend reviewing relevant prior work and appropriately citing the literature wherever it informs your methodology, analysis, or implementation.

For some tasks there are bonus tasks - to demonstrate that you can go the extra mile (which is an important characteristic of being in our group)! We encourage you to try those bonus tasks.

To overcome compute constraints:

- You can use Google Colab, Kaggle for compute intensive jobs.
- Take a subset of data if the dataset is huge - ex. 25% / 50% / 75% of train data

You must attempt this task on your own. Please make sure you cite any external resources that you may use. We will also check your submission for plagiarism, including ChatGPT :)

Submission:

- Put all your code/notebooks in a GitHub repository. Maintain a README.md explaining your codebase, the directory structure, commands to run your project, the dependency libraries used and the approach you followed.
- A Python notebook is mandatory and must clearly log all experiments and results. Submissions consisting only of Python scripts without accountable, logged results will not be considered. All outcomes must be transparent, reproducible, and traceable within the notebook.
- The link to the GitHub repository will be asked for during the interview.
- Presentation / Report:
    - Programming Task : Document the process and compile a presentation / report summarizing your findings, methodologies, and insights gained from the analysis in a presentation/report. Your report must contain your methodology, findings, results etc. On the first page, It must detail exactly what parts of the task you did, and what parts of the task you did not do and why. The report would be one of the main documents that would help us take your application forward for the next stages.

Evaluation:

- You will be evaluated based on how you approach the problem, and not so much on the performance measures like accuracy etc. Although, we would expect you to beat random performance.
- How you present your code and findings. You should justify all the choices you make regarding data, model, and hyperparameters that you use. You should also be able to demonstrate theoretical understanding of the approaches used.
- Across all the tasks, your experiments will give you some quantitative measures to indicate the efficacy of your approach (Accuracy, Recall, MAE, MSE etc) - but dig deeper to analyze the predictions. Can you come up with any hypothesis for why your approach fails/succeeds? Can this analysis help you improve on your approach? Creativity in this analysis is what we are looking for!! - SURPRISE US!!.
- How do you handle and sample from large real-world datasets, and solve tasks with whatever resources you have access to..

For any doubts regarding this task please directly email to the address provided for each task, add the following email in cc - george.rahul@research.iiit.ac.in and debangan.mishra@students.iiit.ac.in. If you don't get a reply within 24 hrs feel free to drop a reminder.

NLP - The Ghost in the Machine

"Le style, c'est l'homme même" (The style is the man himself). — Georges-Louis Leclerc

Task 0: The Library of Babel

You will build a dataset where the primary variable is authorship, not topic.

1. Class 1: Download novels from two authors in Project Gutenberg (e.g., Dickens, Not Shakespeare, etc.). Make sure to clean up your data before using it anywhere.
2. Topic Extraction: Identify 5-10 core topics from the book (e.g., "The Ethics of Science," "The Loneliness of the Arctic").
3. Class 2: Use the Gemini API to generate 500 paragraphs (100-200 words each) on those same topics.
4. Class 3: Use the Gemini API to generate 500 paragraphs on those same topics, specifically prompted to mimic your chosen author's unique style.

Task 1: The Fingerprint

Before training, you must prove that these classes are mathematically distinct. Perform the following analyses:

1. Lexical Richness:
    - Type-Token Ratio (TTR): (Unique words / Total words).
    - Hapax Legomena: Count the number of words that appear only *once* in a 5,000-word sample. (Higher in humans, usually).
2. Syntactic Complexity (SpaCy/NLTK):
    - POS Distribution: Calculate the ratio of Adjectives to Nouns. Does the AI "over-describe" compared to the Human?
    - Dependency Tree Depth: Use SpaCy to calculate the average depth of the sentence parse trees. Longer, more nested branches indicate higher complexity.
3. Punctuation Density: Create a heatmap of punctuation usage (semicolons, em-dashes, exclamation marks).
4. Readability Indices: Calculate the Flesch-Kincaid Grade Level.

Task 2: The Multi-Tiered Detective

Build three detectors to separate AI generated text from Human written ones. Note: If your models fail to reach high accuracy, *this is still a valid research finding if you are able to do good analysis.* Document why.

- Tier A (The Statistician): An XGBoost/Random Forest model using *only* the numerical features from Task 1.
- Tier B (The Semanticist): A Feedforward NN using averaged pre-trained embeddings (GloVe/FastText).
- Tier C (The Transformer): Fine-tune a distilbert-base-uncased or roberta-base using LoRA.

Negative Results: If your models cannot distinguish between the Classes, find out why instead. (e.g., 50/50 accuracy).

Task 3: The Smoking Gun

We need to know *why* the model thinks a text is AI generated.

- Saliency Mapping: Use a library like SHAP or Captum to highlight the words in an "Imposter" paragraph that most strongly signaled "AI" to your Tier C model.
- The Findings: Does the model pick up on specific "AI-isms" (e.g., words like "tapestry," "delve," "testament") or is it looking at the rhythm of the sentence?
- Error Analysis: Find 3 samples where the Human was labeled as AI. Was the author being particularly repetitive? Was the AI being particularly brilliant?

Task 4: The Turing Test

1. The Super-Imposter: Can you "evolve" a paragraph that bypasses your best detector? You will implement a Genetic Algorithm (GA) to optimize a piece of AI-generated text until your classifier labels it as "Human."
   The GA Workflow:

    1. Initial Population: Generate 10 "Imposter" paragraphs using Gemini.
    2. Fitness Function: The "Human" probability score from your model.
    3. Selection: Keep the top 3 paragraphs that look "most human" to the model.

4. Mutation (LLM-as-Mutator): For the next generation, prompt Gemini to "perturb" the winners:
    - *"Rewrite this paragraph to change the rhythm of the sentences while keeping the vocabulary."*
    - *"Introduce a subtle grammatical inconsistency or a rare archaic word."*
5. Iteration: Run this for 5-10 generations.
6. The Goal: Can you reach a >90% "Human" confidence score for a machine-written paragraph?

2. The Personal Test: Take your Statement of Purpose (SOP) or a recent essay you wrote. Run it through your detector.
    - If it says you are AI, why? Try to "humanize" your own writing manually to lower the AI score.